

Exercise 2: ASP.NET Core Web API with Swagger and Postman

Scenario

Modern applications communicate through APIs that allow different systems to exchange information over HTTP.

ASP.NET Core Web API provides a framework for building RESTful services, while Swagger helps developers document, explore, and test API endpoints directly from a browser.

In this exercise, I created an ASP.NET Core Web API application, configured Swagger for API documentation and testing, implemented GET and POST operations using an Employee controller, tested the endpoints using both Swagger UI and Postman, and modified the route configuration to demonstrate custom routing.

Objective

The objectives of this exercise were:

1. Create an ASP.NET Core Web API application.
2. Install and configure Swagger using Swashbuckle.AspNetCore.
3. Implement GET and POST API endpoints.
4. Use ProducesResponseType attributes.
5. Test API endpoints using Swagger UI.
6. Test API endpoints using Postman.
7. Modify controller routing using the Route attribute.
8. Verify API accessibility after route modification.

Project Structure

The following project structure was used:

```
SwaggerDemoApi
├── Controllers
│   └── EmployeeController.cs
├── Models
│   └── Employee.cs
├── Properties
│   └── launchSettings.json
├── Program.cs
├── appsettings.json
├── appsettings.Development.json
└── SwaggerDemoApi.http
```

Implementation Steps

Step 1: Create ASP.NET Core Web API Project

I created a new ASP.NET Core Web API project using Visual Studio Community Edition.

The project automatically generated the basic application structure including:

- Program.cs
- Controllers folder
- Properties folder
- launchSettings.json
- Configuration files

This provided the foundation required for building and testing Web API endpoints.

Step 2: Install Swagger Package

To enable API documentation and testing capabilities, I installed the following NuGet package:

Swashbuckle.AspNetCore

Purpose

This package provides:

- Swagger document generation
- Swagger UI
- API metadata generation
- Interactive API testing

Step 3: Configure Swagger Services

I configured Swagger services inside Program.cs.

```
builder.Services.AddEndpointsApiExplorer();
```

```
builder.Services.AddSwaggerGen();
```

Explanation

AddEndpointsApiExplorer()

This method:

- Discovers API endpoints automatically.
- Generates endpoint metadata.

- Provides information required by Swagger.

AddSwaggerGen()

This method:

- Creates Swagger/OpenAPI documentation.
- Generates endpoint definitions.
- Produces the Swagger JSON file.

Step 4: Configure Swagger Middleware

I enabled Swagger middleware in the application pipeline.

```
app.UseSwagger();
```

```
app.UseSwaggerUI();
```

Explanation

UseSwagger()

Generates the OpenAPI specification document.

UseSwaggerUI()

Provides a browser interface that allows:

- Viewing API endpoints
- Executing requests
- Viewing responses
- Testing APIs without external tools

Step 5: Create Employee Model

I created an Employee model to represent employee information.

Example properties included:

```
public class Employee
{
    public int Id { get; set; }

    public string Name { get; set; }

    public string Department { get; set; }
```

```
public decimal Salary { get; set; }
```

```
}
```

Explanation

The Employee model defines the structure of employee data exchanged between the client and the API.

Step 6: Create Employee Controller

I created an EmployeeController to expose employee-related endpoints.

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class EmployeeController : ControllerBase
```

```
{
```

```
}
```

Explanation

ApiController

Provides:

- Automatic request validation
- API-specific behavior
- Improved error responses

Route

Defines the URL pattern used to access controller actions.

Step 7: Implement GET Action

I implemented a GET action to retrieve employee data.

```
[HttpGet]
```

```
[ProducesResponseType(StatusCodes.Status200OK)]
```

```
public IActionResult GetEmployees()
```

```
{
```

```
    return Ok(employees);
```

```
}
```

Explanation

This action:

- Responds to HTTP GET requests.
- Returns the employee collection.
- Returns HTTP Status Code 200.

ProducesResponseType

The attribute:

```
[ProducesResponseType(StatusCodes.Status200OK)]
```

documents the response type and status code in Swagger.

Benefits include:

- Better API documentation
- Improved client understanding
- Clear response expectations

Step 8: Implement POST Action

I implemented a POST action to add new employee records.

```
[HttpPost]
```

```
[ProducesResponseType(StatusCodes.Status201Created)]
```

```
public IActionResult AddEmployee(Employee employee)
```

```
{  
    employees.Add(employee);  
  
    return Created("", employee);  
}
```

Explanation

This action:

- Accepts employee data from the request body.
- Adds the employee to the collection.

- Returns HTTP Status Code 201 Created.

Step 9: Execute and Test API Using Swagger

I executed the application and opened Swagger UI using:

`https://localhost:<port>/swagger`

Swagger displayed:

- API title
- API version
- Controller endpoints
- Available HTTP methods

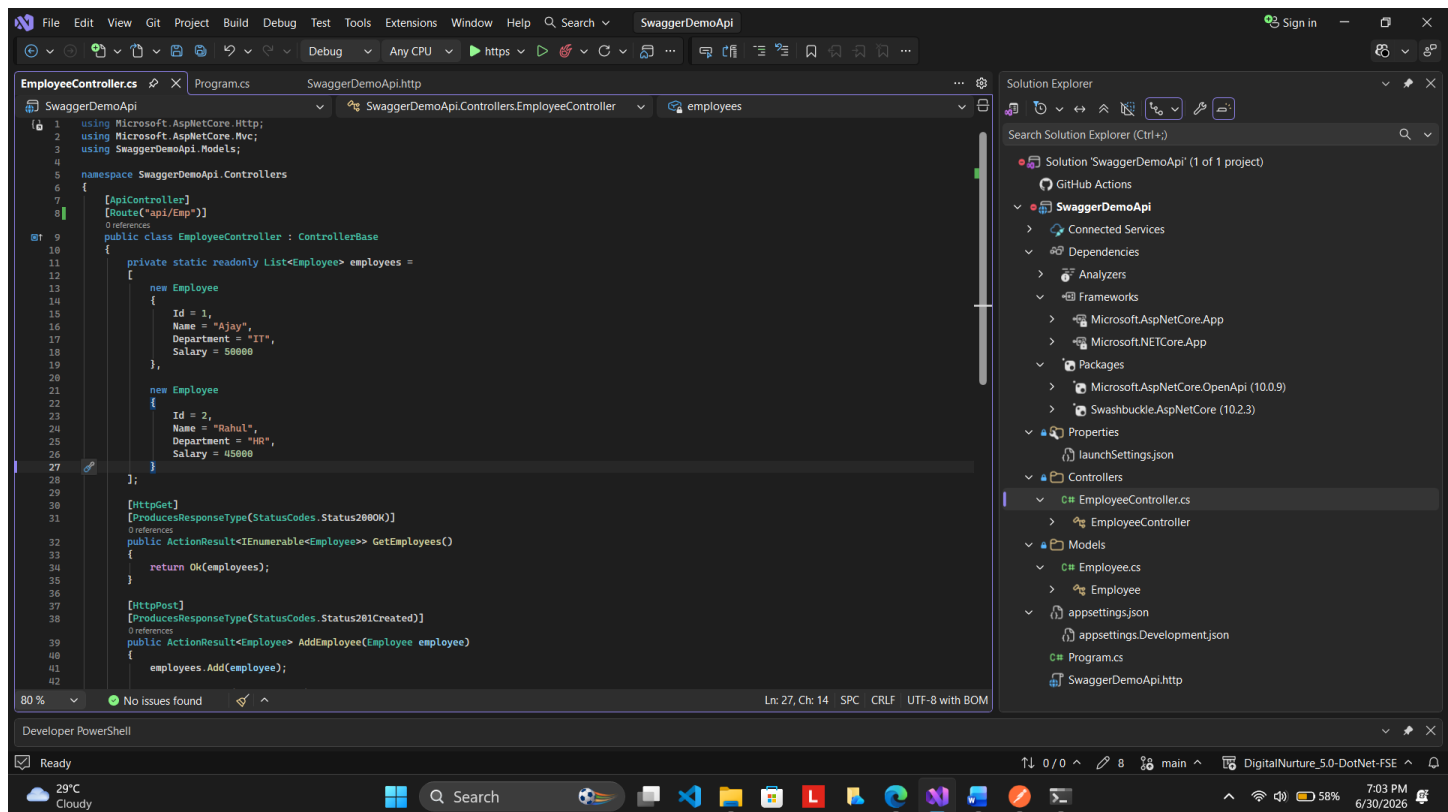
The GET endpoint was executed using the "Try it out" feature.

The employee list was returned successfully.

Screenshot 1 – Project Setup

This screenshot shows:

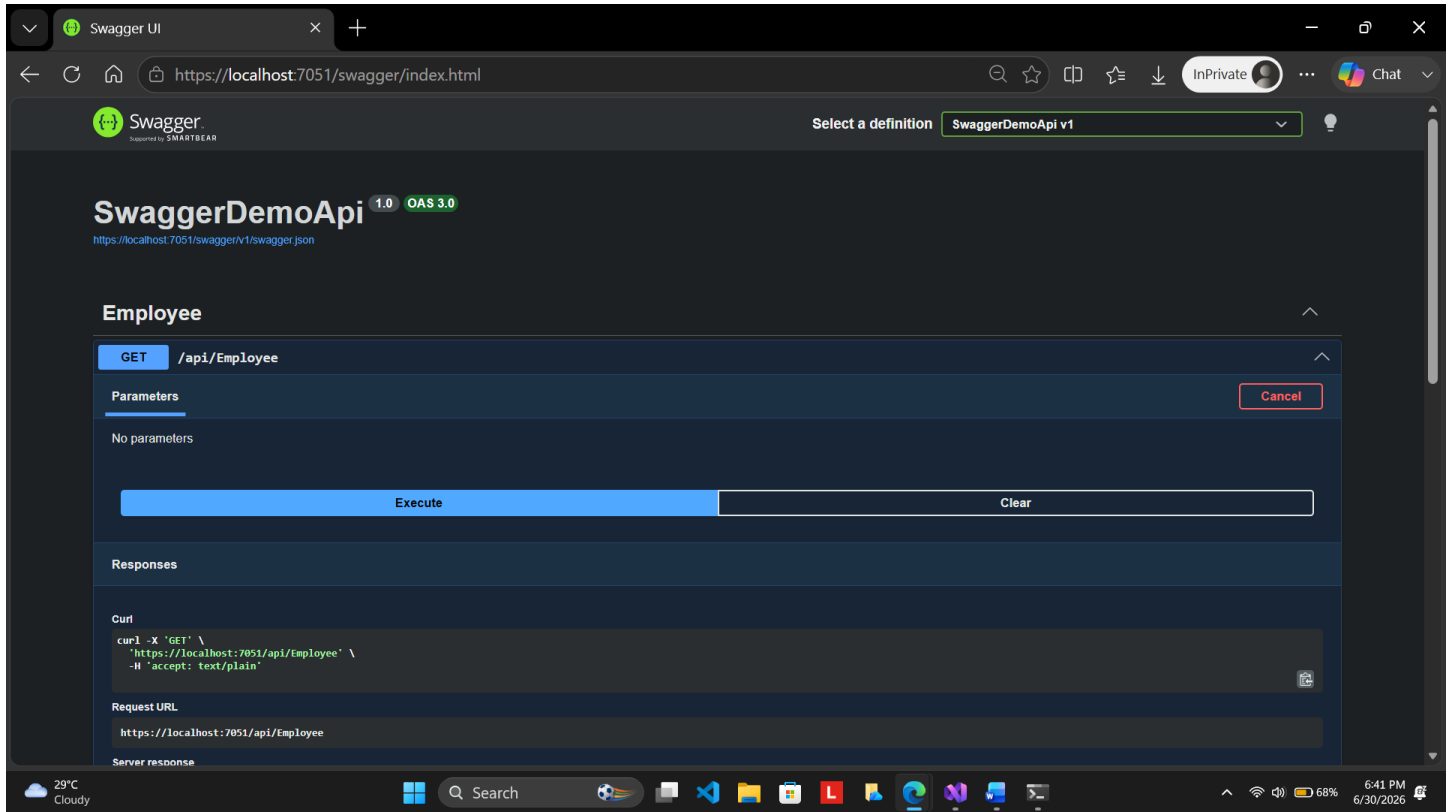
- Solution Explorer
- EmployeeController implementation
- Project structure



Screenshot 2 – Swagger GET Request

This screenshot shows:

- Swagger UI
- GET request execution
- Employee list response



Step 10: Execute POST Request Using Swagger

I tested the POST endpoint through Swagger.

A JSON request body containing employee information was submitted.

Example:

```
{  
  "id": 3,  
  "name": "Priya",  
  "department": "Finance",  
  "salary": 60000  
}
```

Swagger successfully created the new employee record.

The API returned:

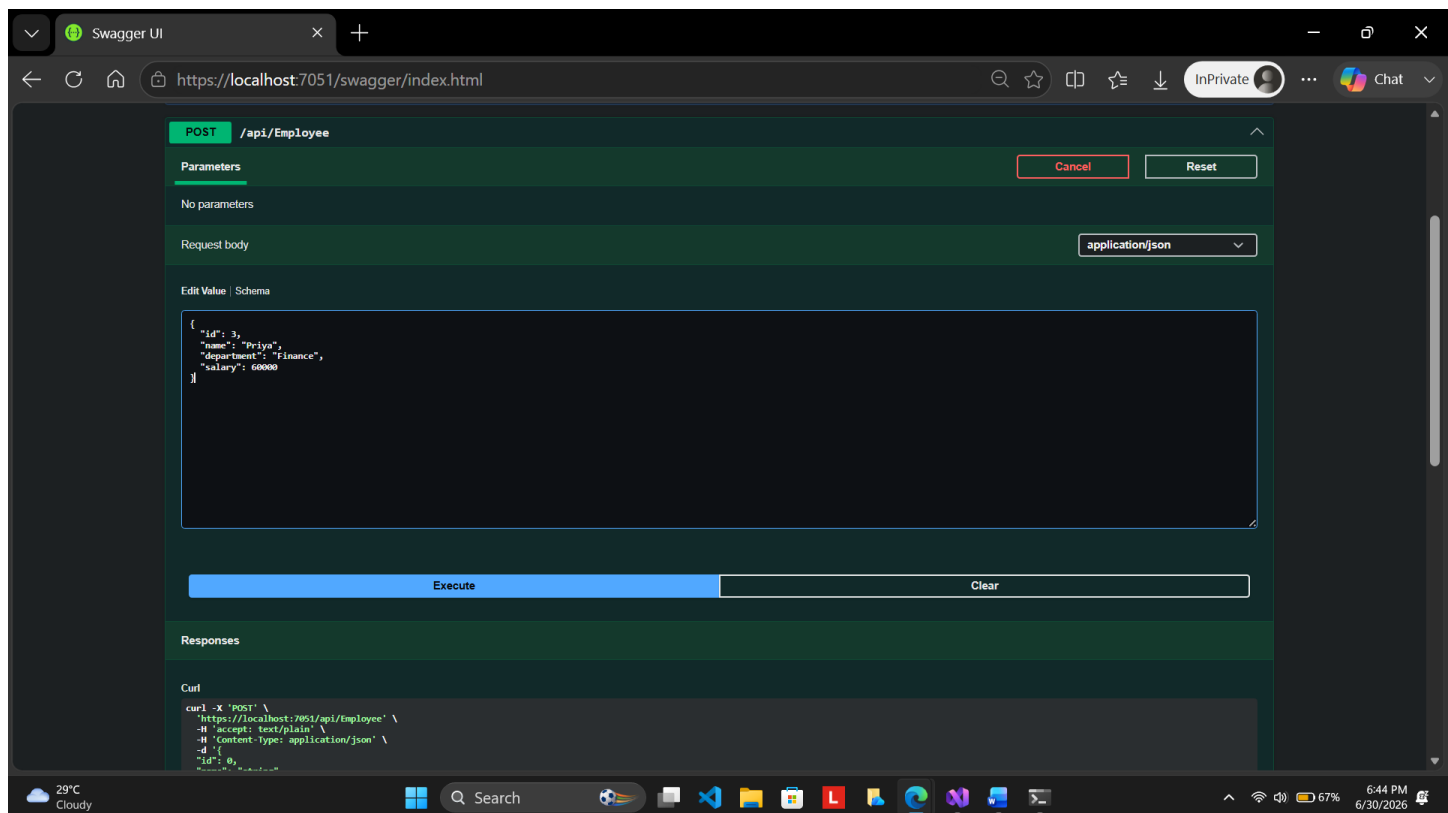
201 Created

indicating successful creation.

Screenshot 3 – Swagger POST Request

This screenshot shows:

- POST endpoint
- Request body
- Employee data submission

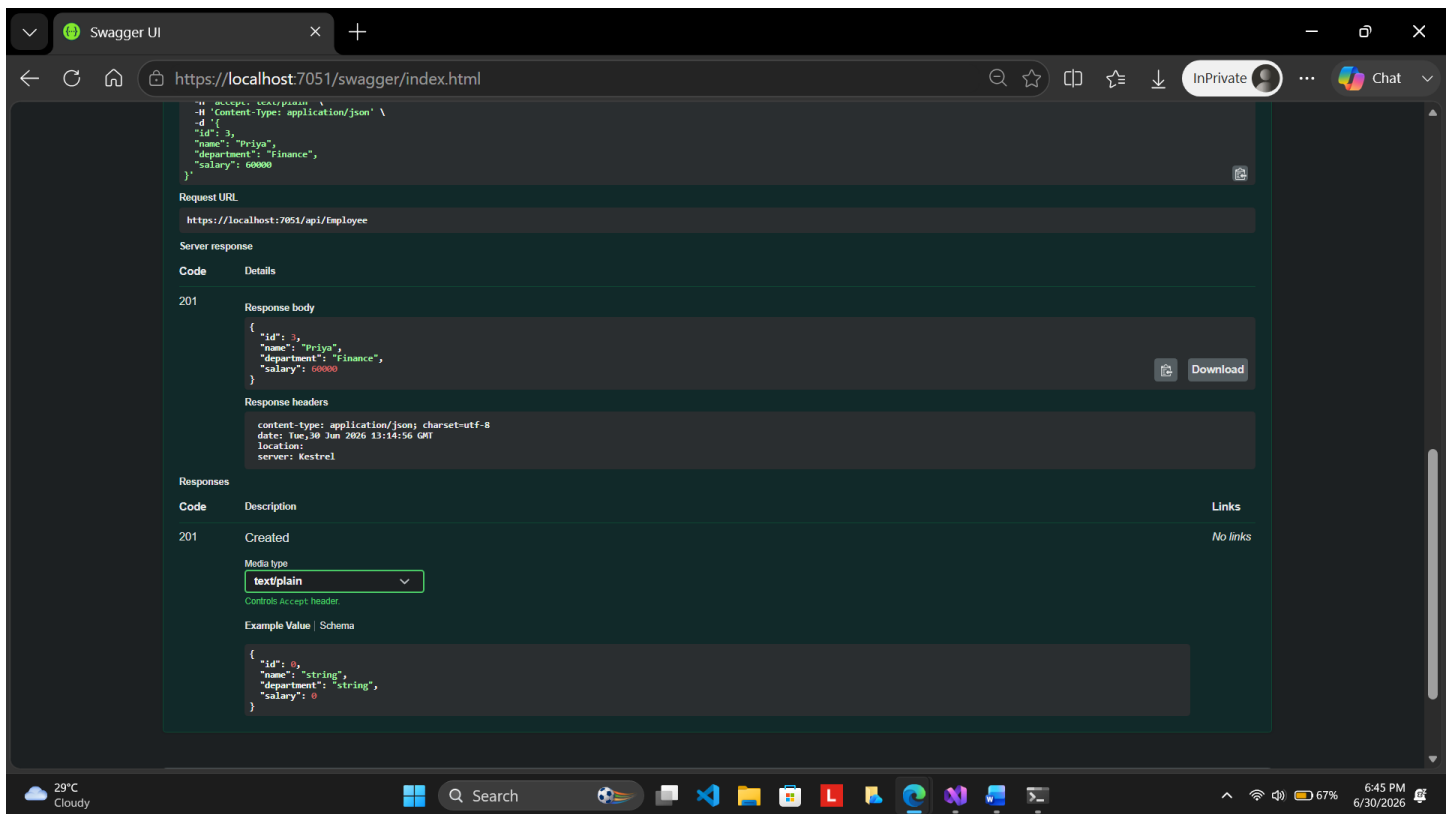


Screenshot 4 – Swagger POST Response

File: swagger_post_response.png

This screenshot shows:

- Successful POST operation
- HTTP Status Code 201 Created
- Created employee record



Step 11: Test API Using Postman

I tested the API using Postman.

Request Details

Method:

GET

URL:

https://localhost:<port>/api/Employee

Verification

I verified:

- Response body
- Employee data
- HTTP Status Code

The request returned:

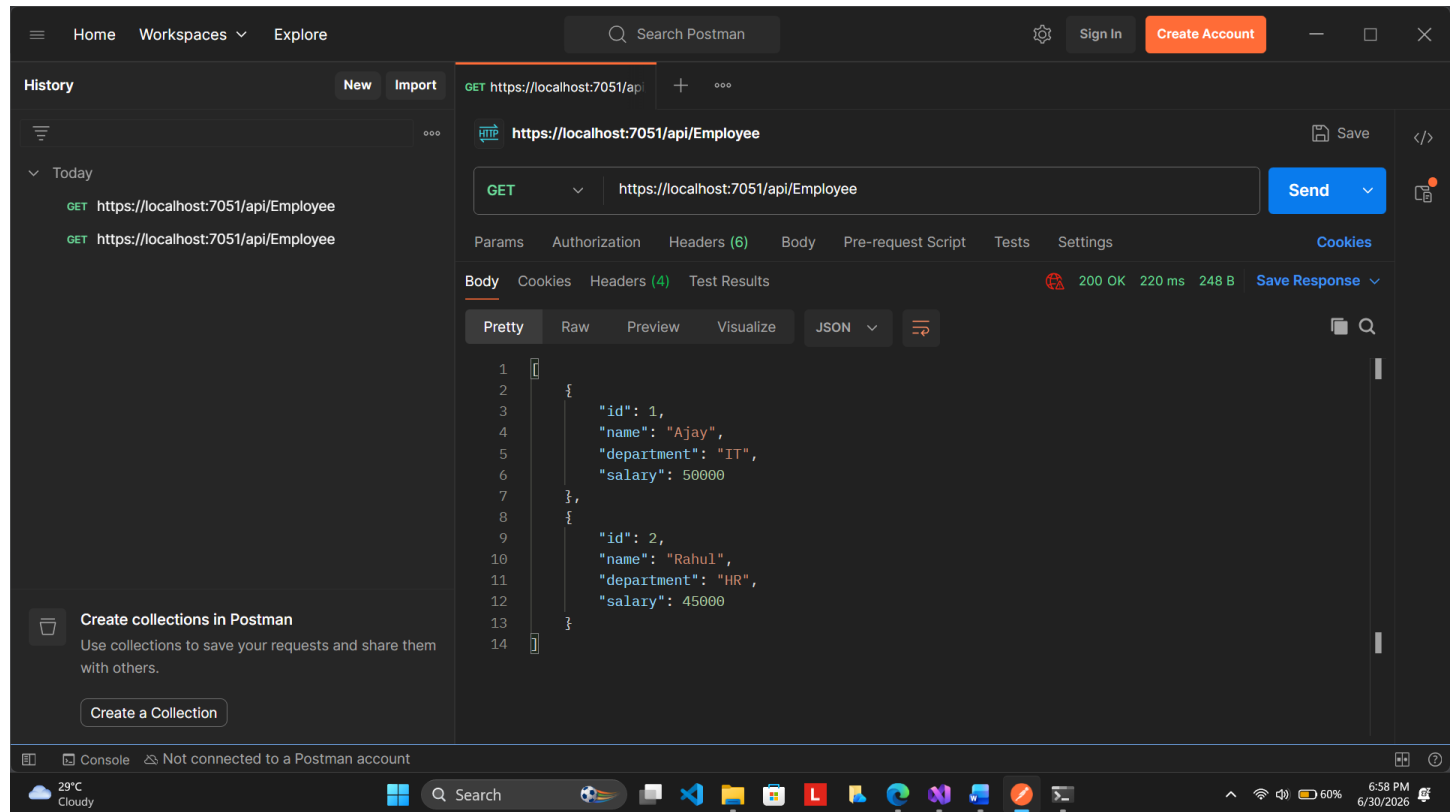
200 OK

confirming successful API execution.

Screenshot 5 – Postman GET Request

This screenshot shows:

- GET request executed through Postman
- Employee list returned successfully
- HTTP Status Code 200 OK



Step 12: Modify Controller Route

To demonstrate route customization, I modified the route attribute.

Before

```
[Route("api/[controller]")]
```

After

```
[Route("api/Emp")]
```

Explanation

This change replaced the default controller name in the URL with a custom route.

Benefits include:

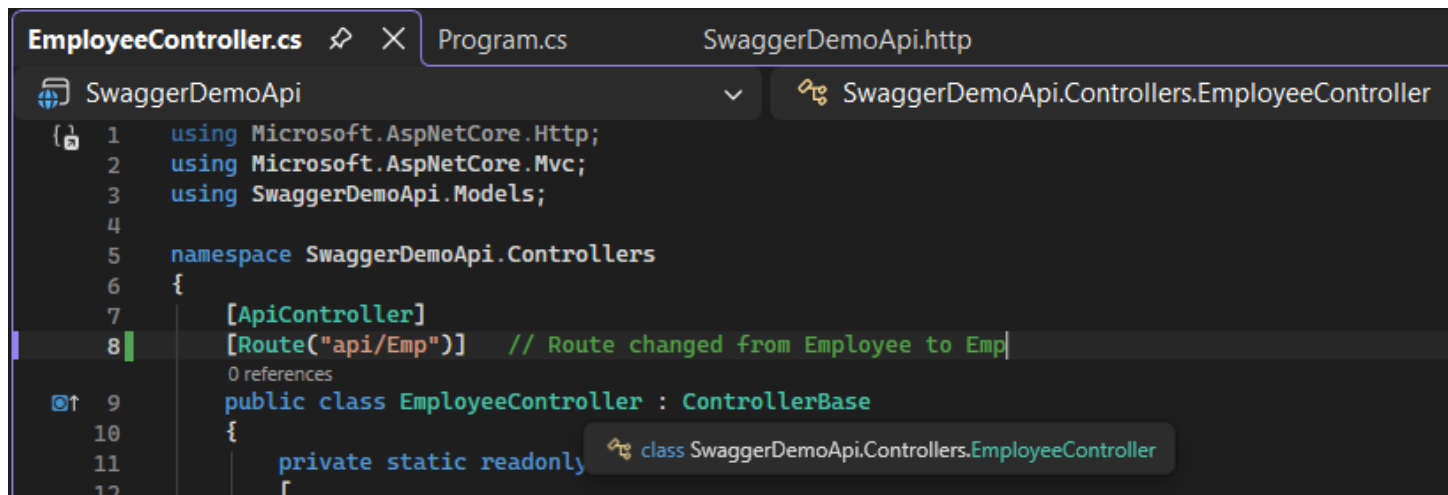
- Cleaner URLs

- User-friendly endpoint names
- Better API design flexibility

Screenshot 6 – Route Modification

This screenshot shows:

- Modified Route attribute
- Route changed from Employee to Emp



The screenshot shows a code editor with the file `EmployeeController.cs` open. The code is as follows:

```
1 using Microsoft.AspNetCore.Http;
2 using Microsoft.AspNetCore.Mvc;
3 using SwaggerDemoApi.Models;
4
5 namespace SwaggerDemoApi.Controllers
6 {
7     [ApiController]
8     [Route("api/Emp")] // Route changed from Employee to Emp
9     public class EmployeeController : ControllerBase
10    {
11        private static readonly
12
```

A tooltip is visible over the `class` keyword on line 9, showing the full class name: `class SwaggerDemoApi.Controllers.EmployeeController`.

Step 13: Verify Modified Route

After modifying the route, I re-executed the application.

The API endpoint became:

`/api/Emp`

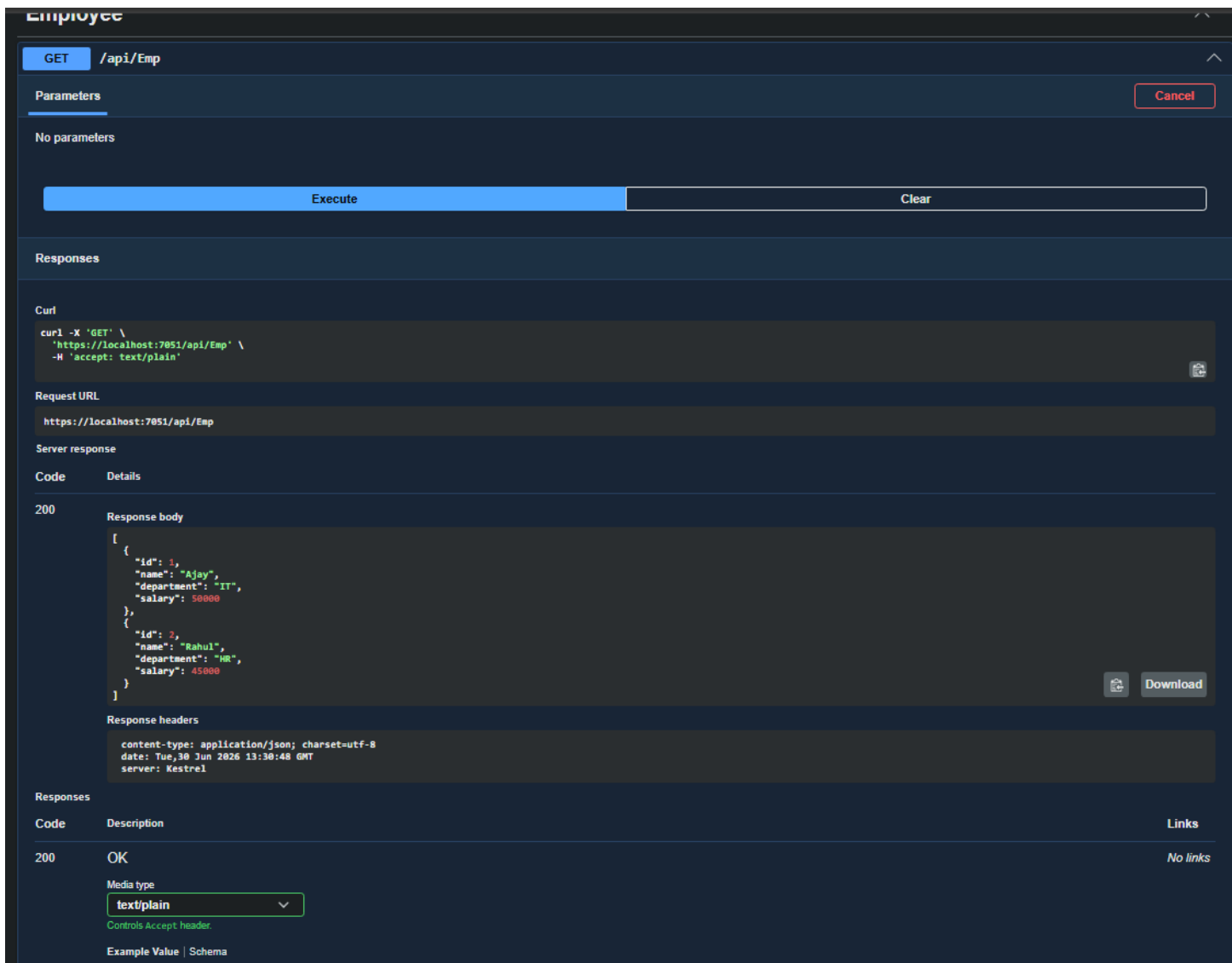
Swagger displayed the updated route successfully.

The endpoint was accessible and returned the expected response.

Screenshot 7 – Route Verification

This screenshot shows:

- Updated route displayed in Swagger
- Successful endpoint access using Emp



ActionName Attribute

The ActionName attribute allows multiple methods to use the same HTTP verb while exposing different logical action names.

Example:

[HttpGet]

[ActionName("GetAllEmployees")]

public IActionResult GetEmployees()

```
{  
    return Ok();  
}
```

Benefits:

- Improves readability
- Supports meaningful operation names
- Helps distinguish similar actions

Although discussed as part of the exercise objectives, no additional implementation was required in this exercise.

Analysis

Swagger Integration

Swagger significantly simplifies API development.

Advantages include:

1. Automatic API documentation.
2. Interactive endpoint testing.
3. Built-in request and response visualization.
4. Faster API validation.

ProducesResponseType Usage

ProducesResponseType improves API documentation by explicitly declaring response codes.

Benefits:

- Better Swagger documentation.
- Easier API consumption.
- Clearer response expectations.

Route Customization

Route customization allows developers to define meaningful endpoint URLs.

Example:

/api/Emp

instead of:

/api/Employee

This improves API usability and readability.

Postman Testing

Postman is a powerful API testing tool.

It enables:

- GET requests
- POST requests
- PUT requests
- DELETE requests
- JSON payload testing
- Header management
- Response inspection

Result

Thus, I successfully created an ASP.NET Core Web API application and configured Swagger using Swashbuckle.AspNetCore.

The GET and POST endpoints were implemented and tested successfully through Swagger UI and Postman. The Route attribute was modified from Employee to Emp, and the updated endpoint was verified successfully. The exercise demonstrated API creation, documentation, testing, and route customization using ASP.NET Core Web API.